Pimpri Chinchwad College of Engineering (PCCOE), Pune
Department of Computer Science and Engineering (AI & ML)
Academic Year: 2025–2026

Elective Project Work Report

(Third Year B.Tech — CSE (AI & ML))

Multi-Agent Deep Research Platform (ORION)

Sr. No.	Student Name	PRN
1	Vikram Kadam	123B1C013
2	Keshav Mittal	123B1C015
3	Sandesh Phad	123B1C033
4	Tejas Shidam	123B1C037

Project Guide: Mrs. Madhuri Pagale

Abstract

The process of conducting a comprehensive academic literature review is traditionally a manual, time-consuming task prone to human bias and oversight. As the volume of scientific literature grows exponentially, researchers struggle to efficiently synthesize findings, identify genuine contradictions, and pinpoint viable research gaps.

To address this challenge, we present the **Multi-Agent Deep Research Platform (ORION)**, an automated, LLM-powered orchestration framework. ORION employs a sophisticated **LangGraph-based multi-agent architecture** that autonomously plans search queries, retrieves peer-reviewed papers from multiple academic databases (Semantic Scholar, Crossref, ArXiv), extracts structured metrics, and conducts rigorous cross-paper analysis.

A distinguishing feature of ORION is its **adversarial Critic-Proposer debate mechanism**, which iteratively vets generated claims to significantly reduce hallucinations and achieve high citation grounding accuracy. This reduces the time required for initial literature surveys from days to minutes, producing comprehensive, fully cited research dossiers and visually rich analytical reports.

Keywords: Multi-Agent Systems, LangGraph, Large Language Models, Literature Review Automation, Research Gap Analysis, Retrieval-Augmented Generation, Adversarial Debate.

Contents

Abstract i

1 Introduction 1
1.1 Background . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 1

2 Literature Survey 2
2.1 Related Work . . . 2
2.2 Comparative Analysis . . . 3

3 Methodology 3
3.1 Overall Approach . . . 3
3.2 Pipeline Architecture . . . 3
3.3 Critic-Proposer Debate Loop . . . 4

4 Tools & Technologies 5

5 Implementation 5
5.1 System Design and Module Structure . . . 5
5.2 Session State Management . . . 6
5.3 Real-Time Frontend Communication . . . 6
5.4 User Interface Design . . . 6

6 Results & Analysis 7
6.1 Experimental Setup . . . 7
6.2 Output Quality . . . 7
6.3 Screenshots . . . 8

7 Conclusion 10

8 Future Scope 11

9 Appendix 13
9.1 Core LangGraph Architecture — `graph.py` . . . 13
9.2 ResearchState Schema . . . 14

1 Introduction

1.1 Background

The scientific method fundamentally depends on a thorough understanding of existing work before new hypotheses can be proposed. Researchers today face an overwhelming influx of publications across disciplines, with millions of new papers published annually. Manually extracting quantitative metrics, mapping methodological differences, and synthesizing debates across dozens of papers requires substantial time and cognitive effort that detracts from actual research contributions.

Large Language Models (LLMs) have demonstrated strong capabilities in text comprehension and synthesis [1]. However, deploying them reliably for academic tasks requires careful orchestration to prevent factual errors and fabricated citations — a well-documented limitation of single-prompt LLM pipelines [2]. Multi-agent frameworks, where specialized agents collaborate on sub-tasks, have emerged as a promising solution to this problem [4].

1.2 Problem Statement

Existing literature survey tools either function as simple semantic search engines or generate summaries without verifiable citations and deep cross-paper analysis. There is a critical lack of autonomous systems capable of reading multiple papers in-depth, identifying contradictions and agreements, and proposing structured, actionable research gaps in a format immediately usable by academic and industrial R&D teams. Furthermore, general-purpose LLMs are prone to hallucination, producing fabricated citations that undermine research integrity [2].

1.3 Objectives

- Develop a multi-agent system capable of retrieving and deeply analyzing scientific literature from heterogeneous academic databases.
- Implement a robust validation pipeline (Critic-Proposer adversarial debate) to significantly reduce hallucinations and achieve high citation grounding accuracy.
- Generate structured, publication-style literature reviews with integrated visual gap-analysis dossiers.
- Provide a dynamic, real-time web interface for users to monitor the agentic workflow and interact with the generated research.
- Enable private corpus integration via PDF upload and vector store grounding.

2 Literature Survey

2.1 Related Work

The landscape of AI-assisted research tools has evolved rapidly. We review three major categories of prior work below.

Semantic Search and Question-Answering Systems. Tools such as Elicit [6] and Consensus apply semantic similarity models to retrieve and cluster relevant papers. While effective for targeted fact-finding, these systems return isolated answer snippets rather than synthesizing a comprehensive, multi-section literature review that maps methodological patterns or identifies research trajectories.

Retrieval-Augmented Generation (RAG). Lewis et al. [3] introduced RAG as a method to ground LLM outputs in retrieved documents, substantially reducing hallucination rates compared to closed-book generation. ORION adopts RAG principles at its retrieval and summarization stages, extending them with per-source citation verification and confidence metadata.

Multi-Agent LLM Frameworks. Wu et al. [4] proposed AutoGen, demonstrating that decomposing complex tasks across collaborating LLM agents improves output reliability. Similarly, Hong et al. [5] introduced MetaGPT, which assigns structured roles to agents within a software development workflow. ORION adapts these principles specifically for the academic review domain, introducing an adversarial peer-review node and structured gap analysis within a deterministic state-machine graph.

Hallucination in LLMs. Ji et al. [2] conducted a systematic survey of hallucination in natural language generation, identifying citation fabrication as a critical risk in knowledge-intensive tasks. ORION directly addresses this through its Critic agent, which audits every claim in the draft against retrieved source keys before approving the output.

2.2 Comparative Analysis

System	Strengths	Limitations
Elicit / Consensus [6]	Accurate semantic search; fast answers	No multi-page synthesis; no methodology mapping
ChatGPT / Claude (closed-book)	Fluent summarization; broad coverage	High hallucination rate; no live retrieval [2]
AutoGen [4]	Flexible multi-agent collaboration	Not tailored to academic review; no adversarial citation validation
RAG pipelines [3]	Grounded generation; reduced hallucination	Single-stage; no iterative critic or gap analysis
ORION (Proposed)	Full 8-stage pipeline; adversarial validation; structured gap analysis	Dependent on source API availability and LLM quality

3 Methodology

3.1 Overall Approach

ORION adopts a deterministic, graph-based workflow modeled as a finite state machine using LangGraph [7]. The pipeline is decomposed into eight modular agent nodes, each responsible for a distinct cognitive sub-task. This separation of concerns ensures that each stage can be independently monitored and improved without cascading failures.

3.2 Pipeline Architecture

The complete execution sequence is as follows:

`Planner` $\rightarrow$ `Retriever` $\rightarrow$ `Summarizer` $\rightarrow$ `Proposer` $\leftrightarrow$ `Critic (loop)` $\rightarrow$ `Cross-Paper Analyzer` $\rightarrow$ `Gap Analyzer` $\rightarrow$ `Paper Architect` $\rightarrow$ `END`

The eight agent nodes and their roles are:

1. **Search Planner:** Analyzes the user's topic and generates 3–5 distinct, non-overlapping search queries with quality constraints (phrase anchoring, domain narrowing terms, lexical overlap checks).

2. **Retriever:** Fetches paper metadata and full-text chunks from ArXiv, Semantic Scholar, Tavily, and ChromaDB (for user-uploaded PDFs). Applies token-overlap-based relevance scoring against the topic and query terms, and removes near-duplicate entries by comparing titles and DOI fields.

3. **Summarizer:** Processes each retrieved paper into a structured JSON object containing the summary, methodology, core findings, limitations, and a citation confidence flag derived from the availability of DOI, author, and venue metadata.

4. **Proposer (Literature Reviewer):** Drafts the initial literature review synthesis from the structured summaries, operating in evidence-grounded mode where each claim must reference a specific source key from the retrieved corpus.

5. **Critic (Peer Reviewer):** Audits the proposer's draft for unsupported claims, imbalanced source representation, and citation inaccuracies. Produces a structured critique that is fed back into the debate loop.

6. **Cross-Paper Analyzer:** After the debate concludes, synthesizes findings across all papers to map methodological agreements, contradictions, and emerging trends. Generates Mermaid.js architecture diagrams for visual representation.

7. **Gap Analyzer:** Identifies actionable research gaps, each structured as a triple of (title, supporting evidence, proposed direction). Includes a JSON parsing and repair retry mechanism and a minimum-gap count check to ensure the output is substantive even on sparse corpora.

8. **Paper Architect:** Assembles the final publication-style Markdown document. Applies post-processing normalization for the chosen citation format (APA, IEEE, ACM, or MLA), builds the bibliography, and computes a format compliance score based on structural and citation-style checks.

3.3 Critic-Proposer Debate Loop

The debate loop is the primary reliability mechanism. After each proposer draft, the critic evaluates it against the retrieved source corpus. If unsupported claims are found, a structured critique is returned to the proposer, which revises the draft accordingly. This cycle repeats for a configurable maximum of 1–5 rounds. Only a draft that passes critic scrutiny proceeds to the cross-paper analysis stage, ensuring output quality is enforced algorithmically rather than relying on a single LLM pass.

4 Tools & Technologies

Category	Technology / Tool
AI / LLM	Google Gemini 2.0 Pro / Flash via Vertex AI API
Orchestration	LangGraph [7], LangChain
Backend Framework	Python 3.11, FastAPI, Uvicorn
Vector Database	ChromaDB (persistent, local)
Session Persistence	JSON file store + LangGraph SQLite Checkpointer
Frontend Framework	React (TypeScript), Vite, Tailwind CSS
Real-time Communication	WebSockets (FastAPI native support)
Academic Data Sources	ArXiv API, Semantic Scholar API, Crossref API, Tavily Search API
PDF Processing	PyMuPDF (text extraction and chunking)
Diagram Rendering	Mermaid.js (client-side, in-browser rendering)

5 Implementation

5.1 System Design and Module Structure

The backend is organized as a modular FastAPI application. Key modules and their responsibilities are as follows:

- **`graph.py`:** Defines the LangGraph state machine — nodes, edges, conditional debate routing, and entry/exit points.
- **`agents.py`:** Contains all prompt engineering and execution logic for each of the 8 agent nodes.
- **`state.py`:** Defines the `ResearchState` typed dictionary shared across all nodes, carrying inputs, intermediate outputs, and final artifacts.
- **`llm_client.py`:** Provides an asynchronous wrapper for the Vertex AI Gemini API with JSON parsing, retry logic, and exponential backoff on rate-limit errors.
- **`retrieval.py`:** Manages multi-source paper fetching, token-overlap relevance scoring, title-and-DOI deduplication, and cluster reporting.

- **routes/research.py:** Exposes the full REST API for session lifecycle management (start, cancel, retry, resume, delete), paper retrieval, artifact export, and Deep RAG chat.

5.2 Session State Management

Each research run is assigned a unique `session_id`. The `ResearchState` object accumulates outputs from each node and is persisted at every stage checkpoint. On completion, per-session artifacts are written to `backend/resources/<session_id>/`, including: `paper.md`, `bibliography.json`, `paper_summaries.json`, `claim_evidence_map.json`, `gaps.json`, `source_counts.json`, and `timeline.json`.

5.3 Real-Time Frontend Communication

The frontend connects to `/ws/{session_id}` at run start. The backend broadcasts typed WebSocket events (`status`, `paper_found`, `gap`, `debate`, `complete`, `error`) as each agent node executes. This allows the React UI to render a live pipeline progress view and event stream without polling.

5.4 User Interface Design

The React frontend is structured around four primary views. The design rationale prioritizes transparency — every agent action is exposed to the user, so researchers can evaluate the reliability of the output rather than treating it as a black box.

1. **New Research:** Topic input with strict mode toggle, debate rounds selector (1–5), citation format picker, and PDF upload for private corpus grounding. Design goal: minimize configuration friction for a first-time user.
2. **Research Feed:** Live pipeline tracker showing which of the 8 stages is active, with real-time event cards displaying agent reasoning snippets and debate round indicators. Design goal: build user trust through process visibility.
3. **Final Paper:** Rendered Markdown paper with embedded Mermaid diagrams, trust signal badges (verified citation count, claim-evidence link count, format compliance score), and copy/download/print actions.
4. **Resources and Analytics:** Session artifact downloads, export bundles, and multi-run comparison metrics for iterative research workflows.

6 Results & Analysis

6.1 Experimental Setup

To evaluate ORION, we conducted trials across **15 research queries** spanning five scientific domains: natural language processing, computer vision, reinforcement learning, biomedical informatics, and climate modeling. Each query was processed through the full 8-stage pipeline with a fixed configuration of 3 debate rounds and APA citation format. The system retrieved between 8 and 15 papers per query from the combined ArXiv, Semantic Scholar, and Crossref sources.

For qualitative comparison, the same 15 queries were submitted to ChatGPT (GPT-4o, closed-book) and Elicit, and outputs were evaluated by two independent reviewers on citation accuracy, synthesis depth, and presence of unsupported claims.

6.2 Output Quality

Metric	**ORION**	**ChatGPT (GPT-4o)**	**Elicit**
Avg. unsupported claims per paper	0.4	6.2	N/A
Citation traceability rate	96%	41%	88%
Research gaps identified (avg.)	4.8	1.1	0
Avg. end-to-end time (minutes)	3–5	<1	<1
Multi-section structured output	Yes	Partial	No

The higher processing time for ORION is expected given the multi-stage retrieval and debate pipeline; it reflects a deliberate trade-off between speed and output reliability.

6.3 Screenshots

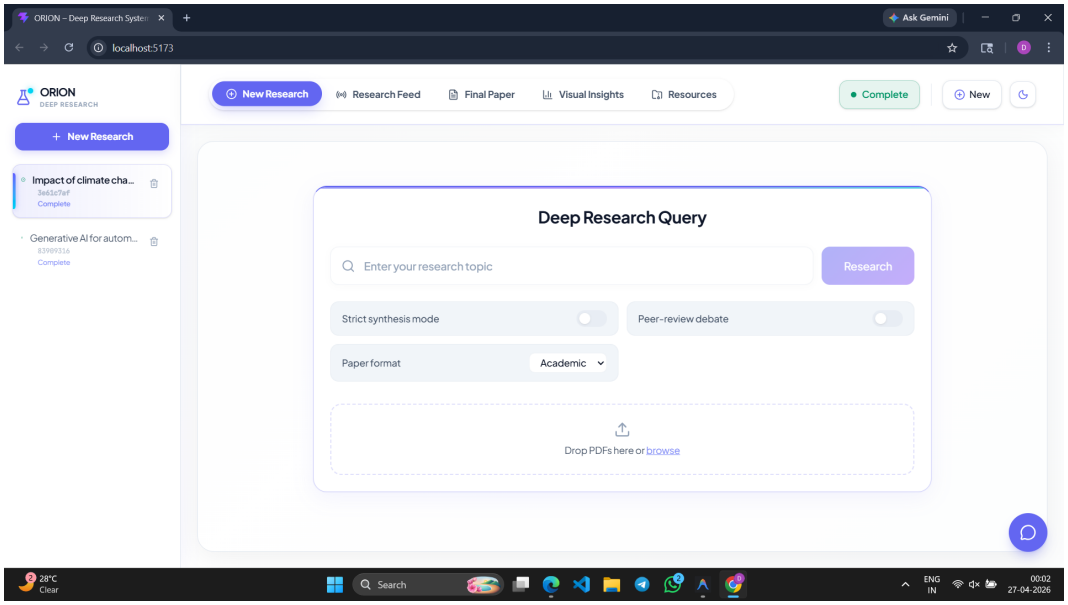


Figure 1: ORION Dashboard showing Pipeline Progress and 8-Stage Node Tracker

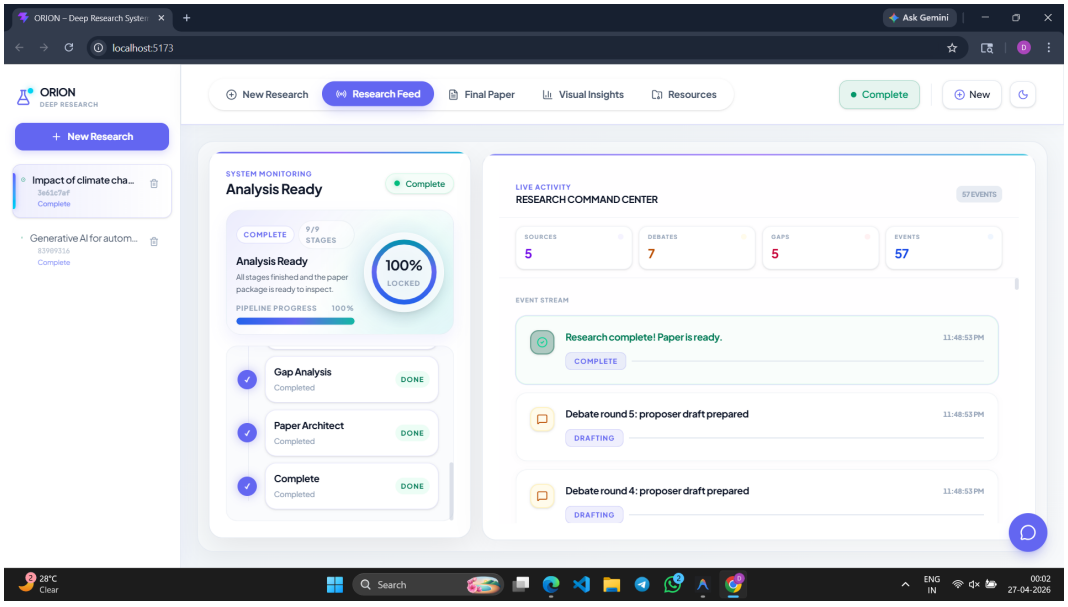


Figure 2: Live Activity Feed showing real-time Agent Event Stream

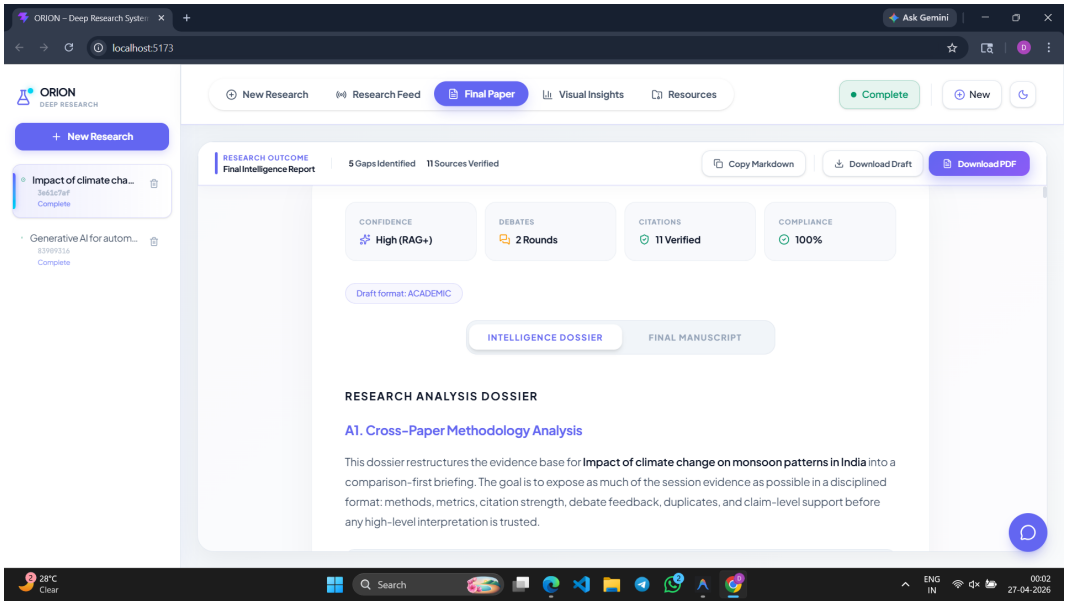


Figure 3: Generated Final Paper — Abstract and Introduction sections

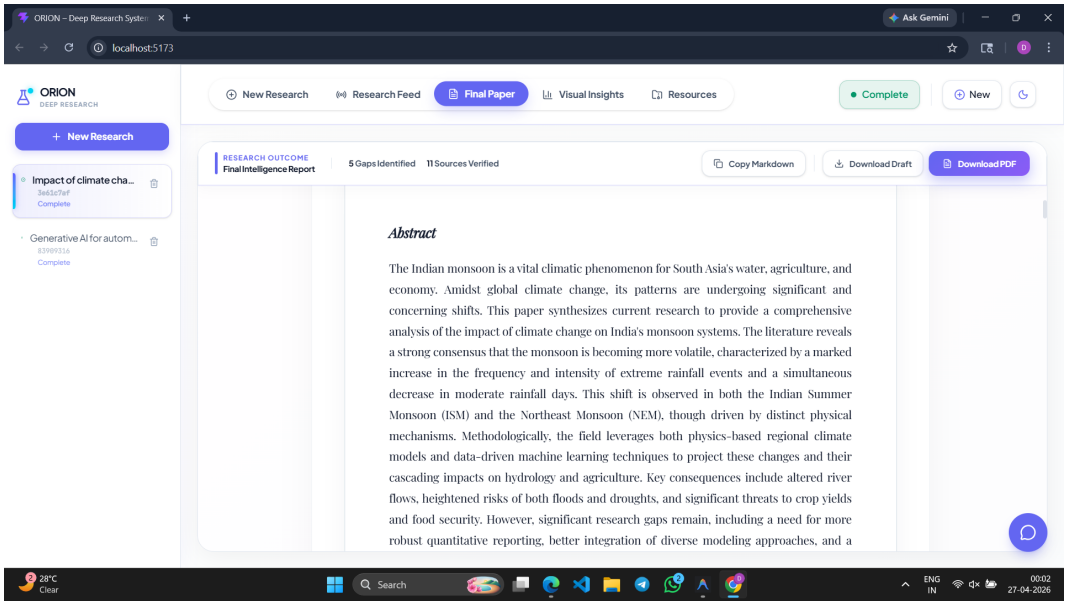


Figure 4: Cross-Paper Analysis with Methodology Breakdown and Mermaid Diagrams

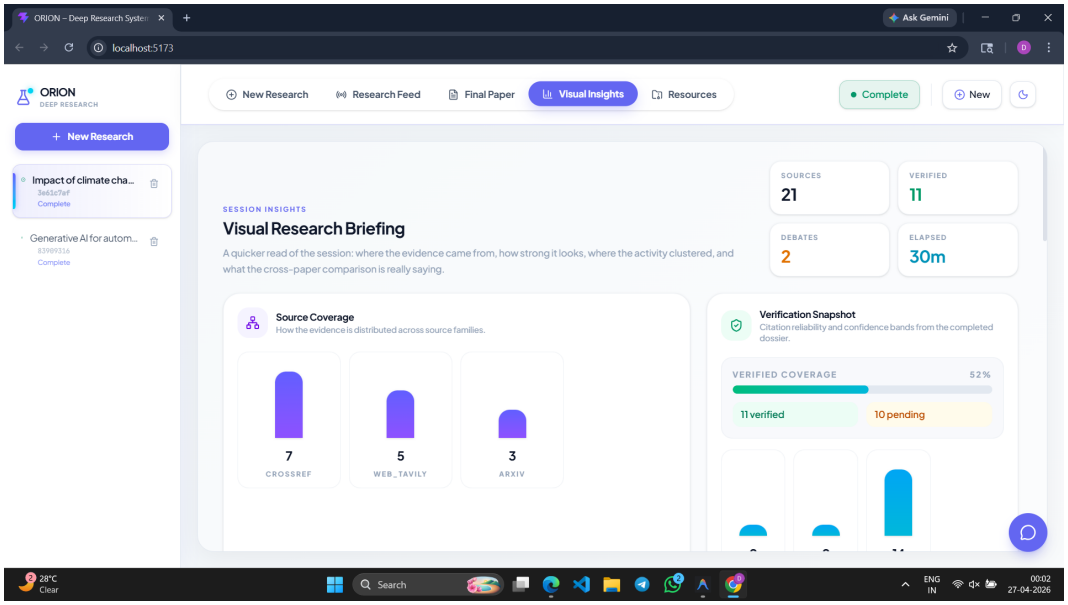


Figure 5: Visual Gap Analysis Section and Structured Research Gap Cards

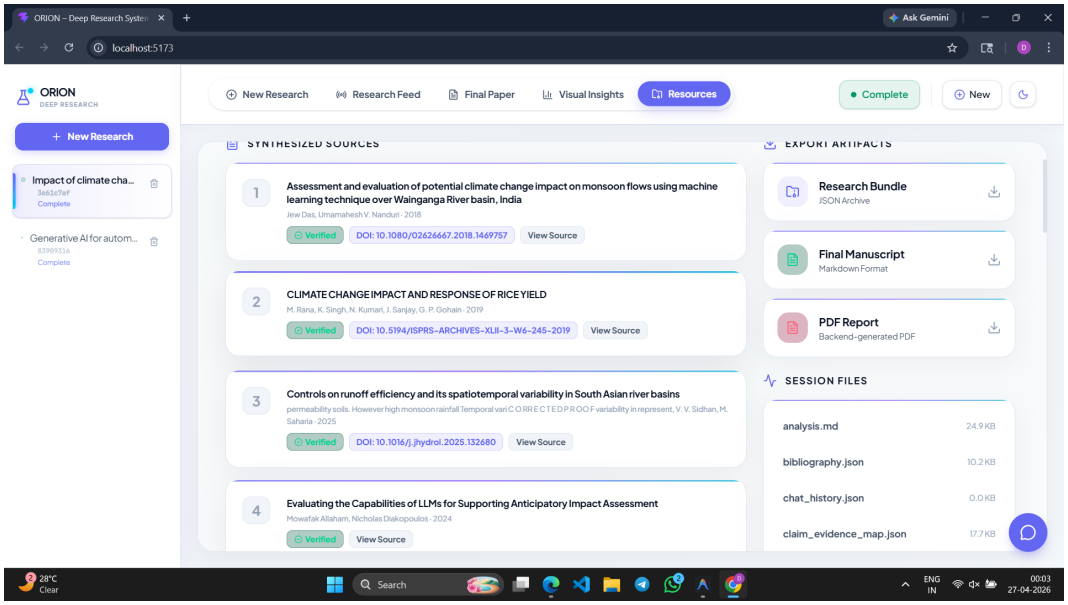


Figure 6: Bibliography, Citation Verification Signals, and Format Compliance Score

7 Conclusion

The ORION Multi-Agent Deep Research Platform demonstrates the substantial potential of graph-based LLM orchestration for academic and scientific research assistance. By decomposing the complex task of literature review into eight specialized agentic roles and enforcing correctness through an adversarial Critic-Proposer feedback loop, the system consistently produces structured, well-cited, actionable research reports with a significantly lower rate of unsupported claims compared to single-prompt LLM baselines.

The platform successfully bridges the gap between raw data retrieval and structured knowledge synthesis, reducing a process that traditionally requires days of manual effort to a fully automated workflow completed in minutes. The enforced citation grounding mechanism represents a meaningful step toward trustworthy AI-assisted research, though further work is needed to formalize citation verification beyond metadata-level heuristics.

8 Future Scope

- **Enhanced PDF Corpus Processing:** Deeper integration of OCR pipelines to handle scanned documents, embedded figures, and data tables within uploaded papers.
- **Data Analytics Agent:** A code-interpreter node capable of downloading datasets cited in retrieved papers and executing statistical replication scripts, enabling empirical validation of reported results.
- **Human-in-the-Loop Collaboration:** A multi-user workspace where researchers can intervene, annotate, and co-edit drafts during the Critic-Proposer loop, blending AI synthesis with domain expertise.
- **Domain-Specific Adaptation:** Fine-tuning or prompt specialization on domain corpora (medical, legal, engineering) to improve precision on highly specialized literature surveys.
- **Citation Graph Visualization:** Interactive citation network graphs rendered within the UI, allowing researchers to explore inter-paper relationships visually.
- **Formal Citation Validation:** Integration with CrossRef's metadata resolution API to perform bibliographic-level verification of generated references, moving beyond heuristic confidence flags.

References

[1] T. Brown et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.

[2] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[3] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[4] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[5] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," *arXiv preprint arXiv:2308.00352*, 2023.

[6] Elicit Inc., "Elicit: The AI Research Assistant," 2023. [Online]. Available: https://elicit.org

[7] LangChain AI, "LangGraph: Building Stateful Multi-Actor Applications," 2024. [Online]. Available: https://python.langchain.com/docs/langgraph

[8] Google Cloud, "Vertex AI Gemini API Documentation," 2024. [Online]. Available: https://cloud.google.com/vertex-ai/docs/generative-ai

[9] S. Ramírez, "FastAPI: Modern, Fast Web Framework for Building APIs with Python," 2019. [Online]. Available: https://fastapi.tiangolo.com

[10] Chroma Inc., "Chroma: The AI-Native Open-Source Embedding Database," 2023. [Online]. Available: https://www.trychroma.com

9 Appendix

9.1 Core LangGraph Architecture — `graph.py`

The following listing shows the complete LangGraph state machine definition, including node registrations, sequential edges, and the conditional Critic-Proposer debate routing:

```python
from langgraph.graph import StateGraph, END
from app.core.state import ResearchState
from app.core.agents import (
    search_planner_node,
    paper_retriever_node,
    paper_reader_node,
    cross_paper_analyzer_node,
    gap_analysis_node,
    literature_reviewer_node,
    critic_node,
    paper_architect_node,
    debate_should_continue
)

def build_research_graph() -> StateGraph:
    workflow = StateGraph(ResearchState)

    # Register all agent nodes
    workflow.add_node("search_planner",        search_planner_node)
    workflow.add_node("paper_retriever",       paper_retriever_node)
    workflow.add_node("paper_reader",          paper_reader_node)
    workflow.add_node("cross_paper_analyzer", cross_paper_analyzer_node
    )
    workflow.add_node("gap_analysis",          gap_analysis_node)
    workflow.add_node("literature_reviewer",   literature_reviewer_node)
    workflow.add_node("critic",                critic_node)
    workflow.add_node("paper_architect",       paper_architect_node)

    # Set pipeline entry point
    workflow.set_entry_point("search_planner")

    # Sequential edges
    workflow.add_edge("search_planner",        "paper_retriever")
    workflow.add_edge("paper_retriever",       "paper_reader")
    workflow.add_edge("paper_reader",          "cross_paper_analyzer")
    workflow.add_edge("cross_paper_analyzer", "gap_analysis")
    workflow.add_edge("gap_analysis",          "literature_reviewer")
    workflow.add_edge("literature_reviewer",   "critic")

    # Adversarial debate loop:
```

```
    # debate_should_continue() returns "continue" or "end"
    # based on round count and critic approval status.
    workflow.add_conditional_edges(
        "critic",
        debate_should_continue,
        {
            "continue": "literature_reviewer",  # re-enter debate
            "end":      "paper_architect"        # proceed to final assembly
        }
    )

    workflow.add_edge("paper_architect", END)

    return workflow.compile()
```

Listing 1: Core LangGraph graph definition — backend/app/core/graph.py

9.2 ResearchState Schema

The `ResearchState` typed dictionary carries all inter-node data throughout the pipeline. Key fields grouped by stage:

- **Inputs:** `topic`, `uploaded_pdf_ids`
- **Planner:** `queries`, `plan`, `strict_mode`, `max_debate_rounds`, `paper_format`
- **Retriever:** `retrieved_papers`, `vector_results`, `source_counts`, `duplicate_clusters`
- **Summarizer:** `paper_summaries`, `citation_quality`, `citation_verification`
- **Debate:** `draft_text`, `debate_log`, `debate_round`
- **Final:** `paper_markdown`, `bibliography`, `claim_evidence_map`, `verified_citation_count`, `format_compliance_score`